# Coding Assignment 1

Name – Vanshika

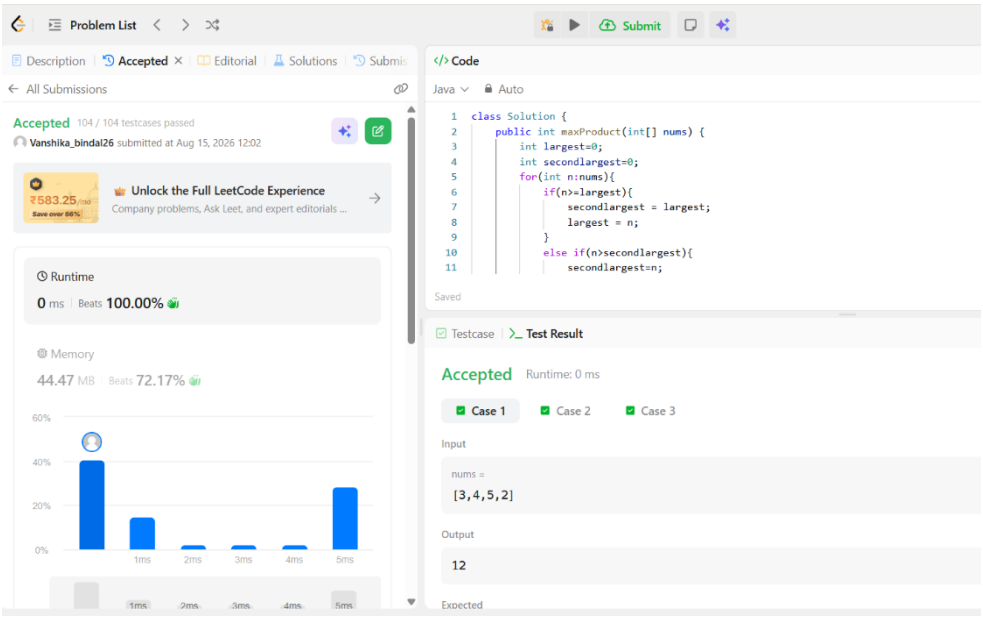
University Roll no. – 12515001755

Class Roll no. – 65

Section – FB

# Question 1: Maximum Product of Two Elements in an Array

```
class Solution {
    public int maxProduct(int[] nums) {
        int largest=0;
        int secondlargest=0;
        for(int n:nums){
            if(n>=largest){
                secondlargest = largest;
                largest = n;
            }
            else if(n>secondlargest){
                secondlargest=n;
            }
        }
        return (largest-1)*(secondlargest-1);
    }
}
```



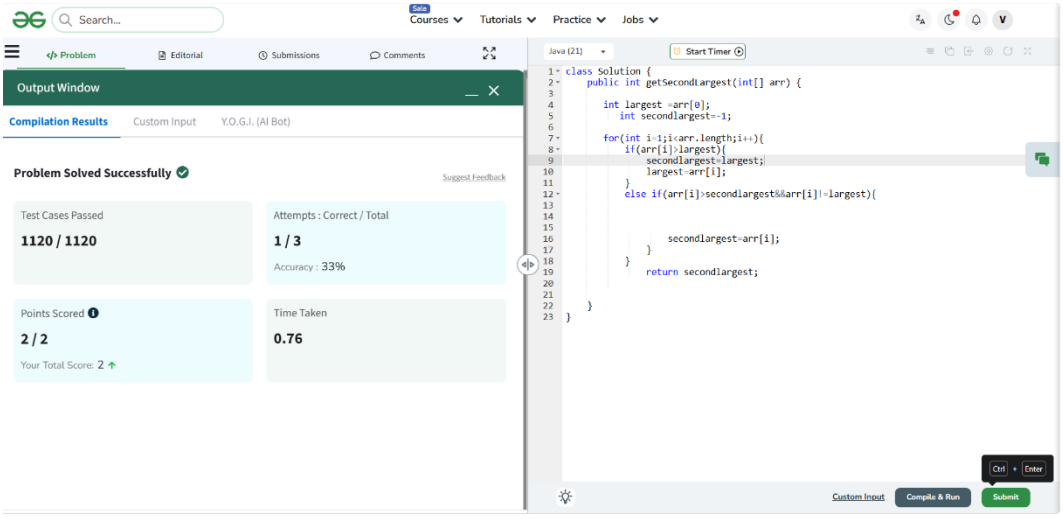
# Question 2: Second-Largest

```
class Solution {
  public int getSecondLargest(int[] arr) {

    int largest =arr[0];
      int secondlargest=-1;

    for(int i=1;i<arr.length;i++){
      if(arr[i]>largest){
        secondlargest=largest;
        largest=arr[i];
      }
      else if(arr[i]>secondlargest&&arr[i]!=largest);

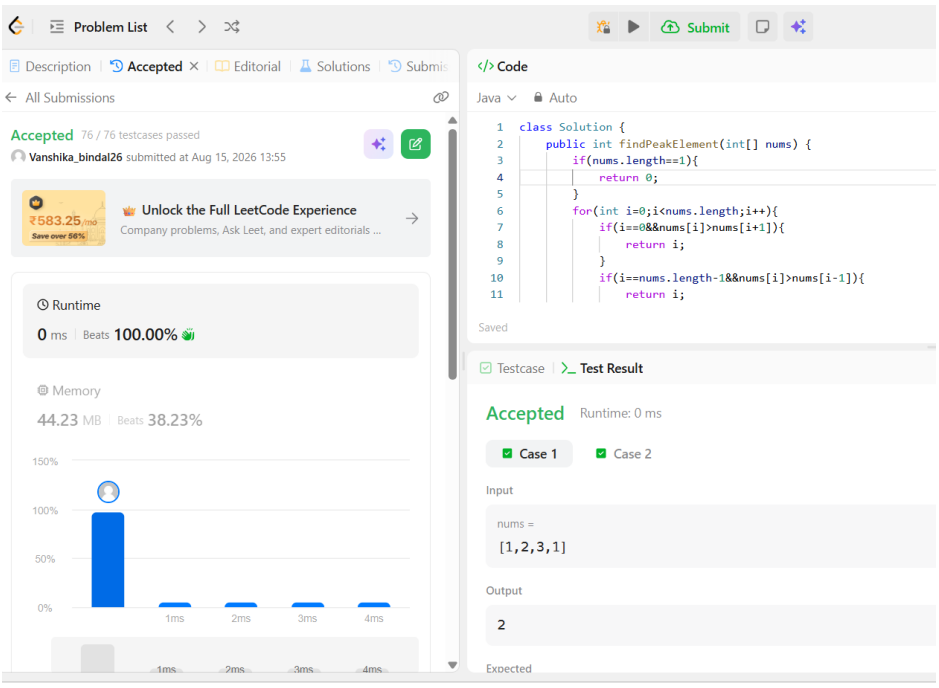
          secondlargest=arr[i];
        }
      }
        return secondlargest;
  }
}
```



# Question 3: Find Peak Element

```
class Solution {
    public int findPeakElement(int[] nums) {
        if(nums.length==1){
            return 0;
        }
        for(int i=0;i<nums.length;i++){
            if(i==0&&nums[i]>nums[i+1]){
                return i;
            }
            if(i==nums.length-1&&nums[i]>nums[i-1]){
                return i;
            }
            if(i>0&&i<nums.length-1&&nums[i]>nums[i+1]&&nums[i]>nums[i-1]){
                return i;
            }

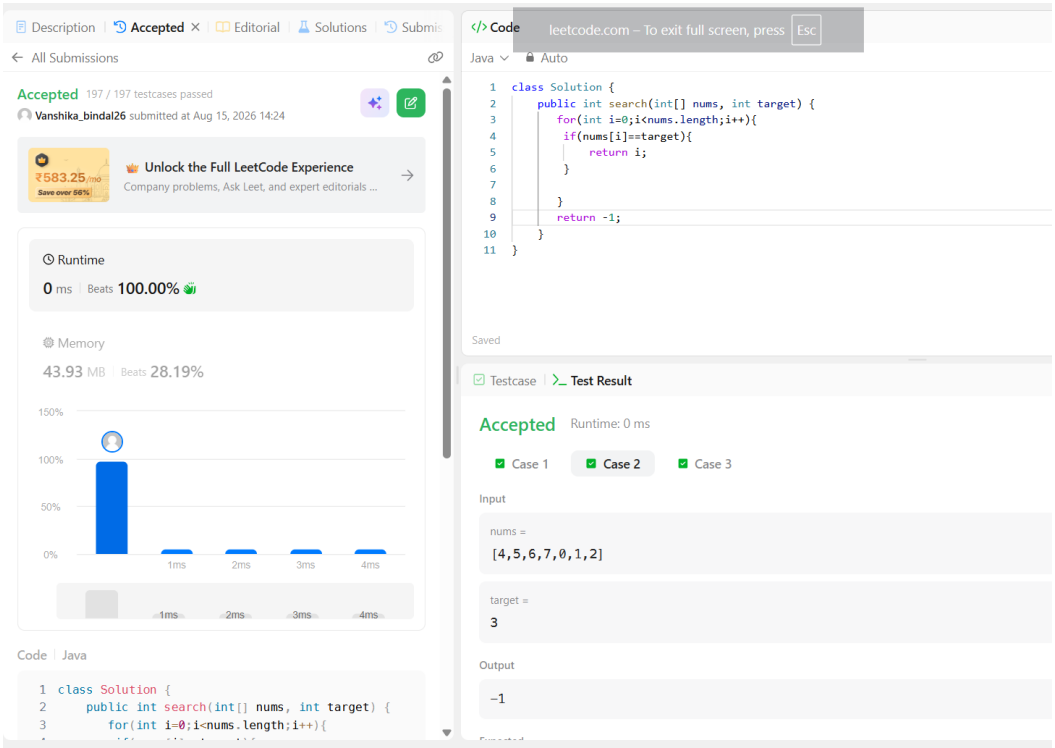
        }
        return 0;
    }
}
```



# Question 4: Search in Rotated Sorted Array

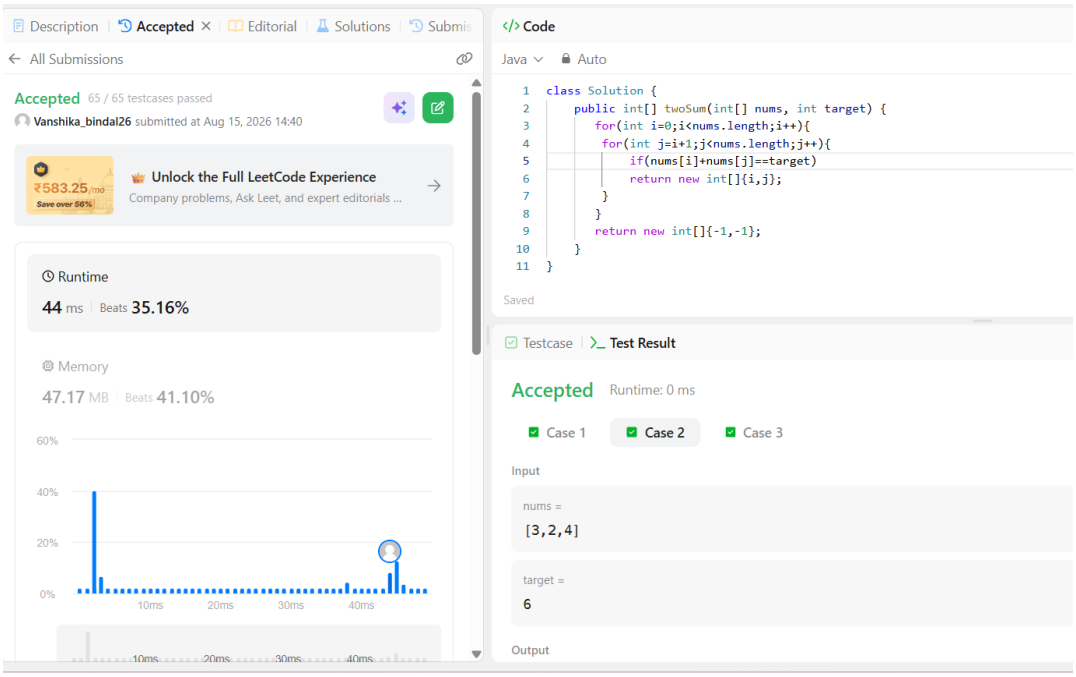
```
class Solution {
    public int search(int[] nums, int target) {
        for(int i=0;i<nums.length;i++){
         if(nums[i]==target){
            return i;
         }

        }
        return -1;
    }
}
```



# Question 5: Two Sum

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        for(int i=0;i<nums.length;i++){
         for(int j=i+1;j<nums.length;j++){
            if(nums[i]+nums[j]==target)
            return new int[]{i,j};
         }
        }
        return new int[]{-1,-1};
    }
}
```



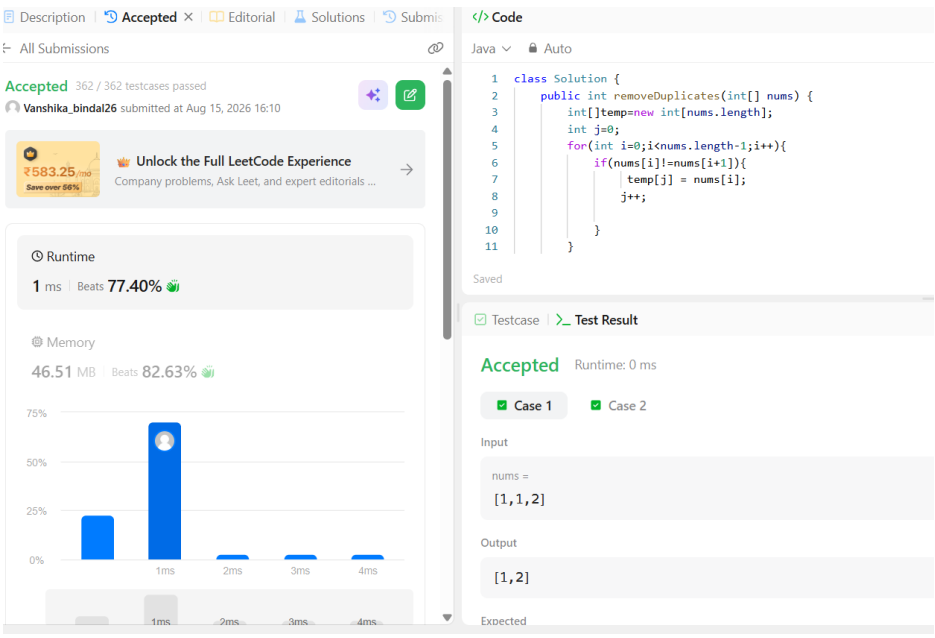
# Question 6: Remove Duplicates from Sorted Array

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int[]temp=new int[nums.length];
        int j=0;
        for(int i=0;i<nums.length-1;i++){
            if(nums[i]!=nums[i+1]){
                temp[j] = nums[i];
                j++;

            }
        }
            temp[j]=nums[nums.length-1];
            j++;
             for (int i = 0; i < j; i++) {
            nums[i] = temp[i];
        }

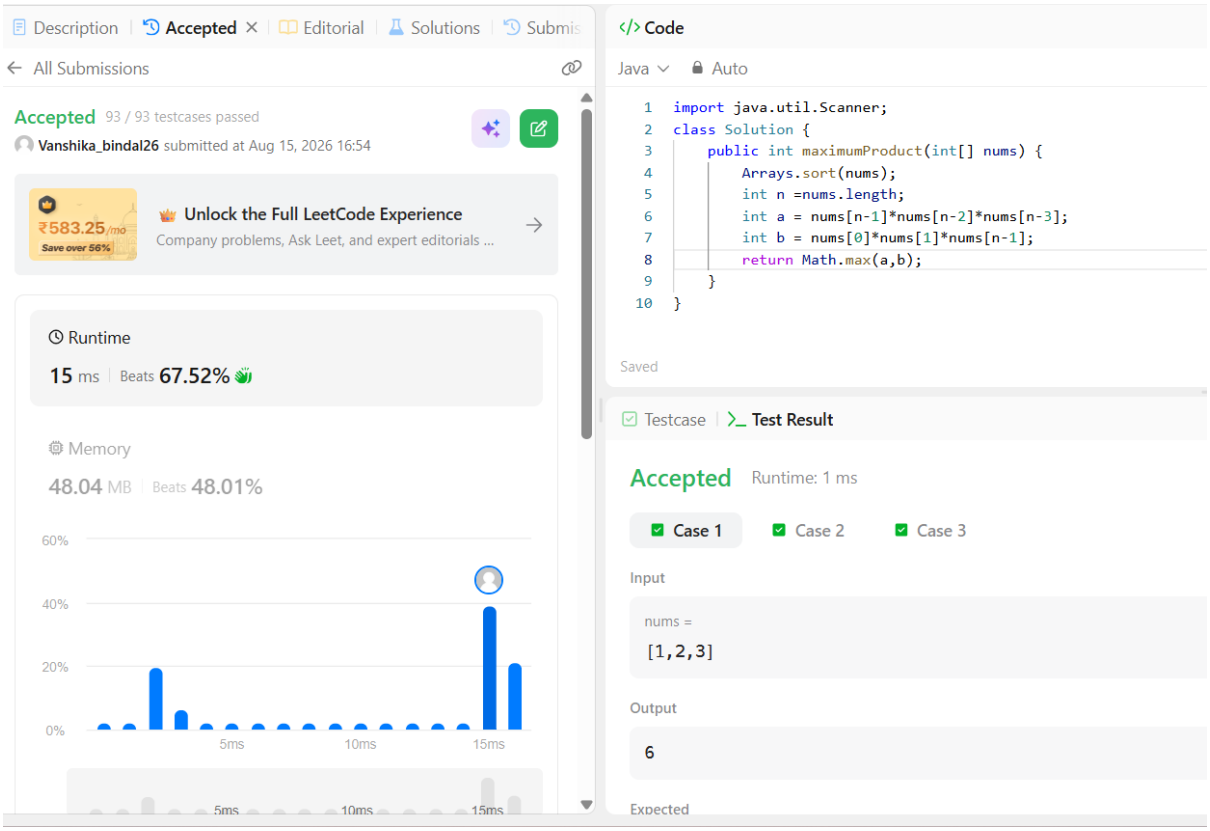
        return j;

    }
}
```



# Question 7: Maximum Product of Three Numbers

```
import java.util.Scanner;
class Solution {
    public int maximumProduct(int[] nums) {
        Arrays.sort(nums);
        int n =nums.length;
        int a = nums[n-1]*nums[n-2]*nums[n-3];
        int b = nums[0]*nums[1]*nums[n-1];
        return Math.max(a,b);
    }
}
```



# Question 8: Best Time to Buy and Sell Stock

```
class Solution {
    public int maxProfit(int[] prices) {
        int buy = prices[0];
        int profit = 0;

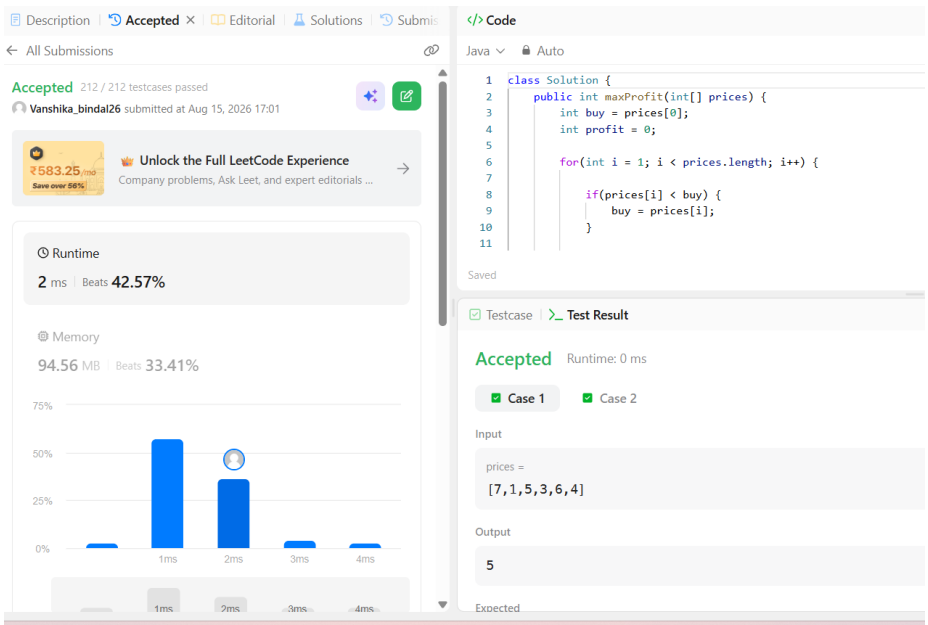
        for(int i = 1; i < prices.length; i++) {

            if(prices[i] < buy) {
                buy = prices[i];
            }

            int currentProfit = prices[i] - buy;

            if(currentProfit > profit) {
                profit = currentProfit;
            }
        }

        return profit;
    }
}
```



# Question 9: Fruits Into Baskets II

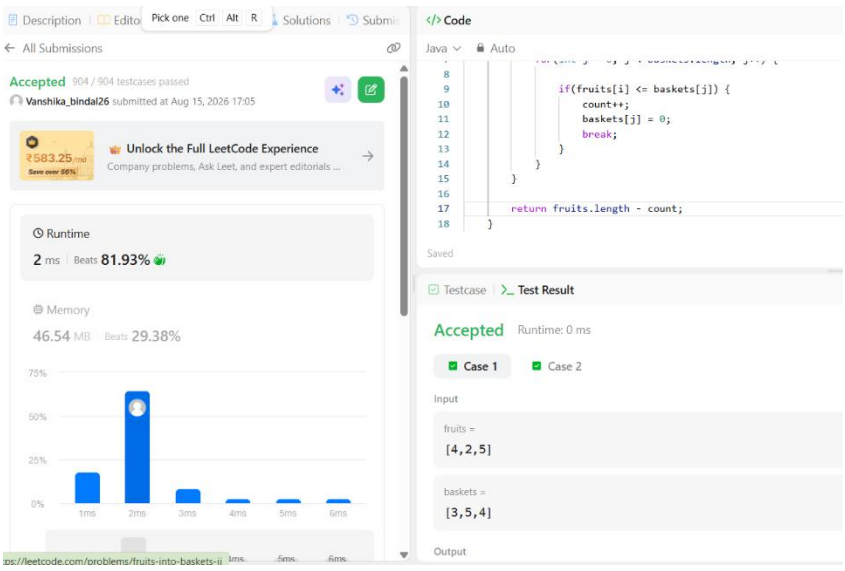
```
class Solution {
    public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
        int count = 0;

       for(int i = 0; i < fruits.length; i++) {

          for(int j = 0; j < baskets.length; j++) {

             if(fruits[i] <= baskets[j]) {
                count++;
                baskets[j] = 0;
                break;
             }
          }
       }

       return fruits.length - count;
    }
}
```



# Question 10: How Many Numbers Are Smaller Than the Current Number

```
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {
        int[] ans = new int[nums.length];

        for(int i = 0; i < nums.length; i++) {

            int count = 0;

            for(int j = 0; j < nums.length; j++) {

                if(nums[j] < nums[i]) {
                    count++;
                }
            }

            ans[i] = count;
        }

        return ans;
    }
}
```

